Efficient Access Scheme for Multi-bank Based NTT Architecture Through Conflict Graph

Xiangren Chen[1], Bohan Yang[1], Yong Lu[2], Shouyi Yin[1], Shaojun Wei[1], Leibo Liu[*] [1]
[1]School of Integrated Circuits, Tsinghua University, China. [2]HEXIN Technologies (Shanghai) Co., Ltd.
Email:liulb@tsinghua.edu.cn

ABSTRACT

Number Theoretical Transform (NTT) hardware accelerator becomes crucial building block in many cryptosystems like post-quantum cryptography. In this paper, we provide new insights into the construction of conflict-free memory mapping scheme (CFMMS) for multi-bank NTT architecture. Firstly, we offer parallel loop structure of arbitrary-radix NTT and propose two point-fetching modes. Afterwards, we transform the conflict-free mapping problem into conflict graph and develop novel heuristic to explore the design space of CFMMS, which turns out more efficient access scheme than classic works. To further verify the methodology, we design high-performance NTT/INTT kernels for Dilithium, whose area-time efficiency significantly outperforms state-of-the-art works on the similar FPGA platform.

KEYWORDS

number theoretical transform, arbitrary radix, multi-bank, conflict-free memory mapping scheme, Dilithium.

1 INTRODUCTION

NTT plays an important role in a lot of cryptographic algorithms, encompassing lattice-based post-quantum cryptography (PQC), homomorphic encryption (HE) and zero knowledge proof. However, the heavy computation overhead restricts the practical hardware implementation. Among these schemes, an integer polynomial is frequently transformed from coefficient into point-value via the NTT operation, which reduces the computation complexity of N-term polynomial multiplication from $O(N^2)$ to $O(N \log N)$. Hence, research focused on NTT hardware accelerator has almost reached a climax phase.

One of the prevailing techniques to pursue high throughput for NTT kernel is memory interleaving, where multiple independent banks are matched with parallel processing elements (PEs). [1] presents the hardware architecture of radix-2 constant geometry NTT with two butterfly units (BFUs) based on four single-port SRAMs. A radix-2 stockham NTT hardware core with 16 parallel BFUs are reported in [3]. The constant geometry NTT owns a simpler access pattern than in-place NTT at the price of double memory overhead. [15] proposes the multi-bank based hardware architecture of radix-2 in-place NTT with two BFUs. Although the advantages in terms of small memory footprint, the varying strides in every stage of in-place NTT make the hardware architecture hard to be parallelized, which becomes more complicated when considering the higher radix NTT with parallel access patterns. Indeed, a well-devised conflict-free memory mapping scheme can eliminate the memory conflicts and avoid pipeline stalls while still consume low logic and memory resource. [6] presents a conflict-free memory addressing strategy for arbitrary radix in-place FFT and reduces the memory storage to N-word vector length. [12] propose a dedicated hardware architecture for radix-16 NTT based on this method. While [15] points out that this method still causes memory conflict when encountering odd number of vector length in radix-2 NTT equipped with multiple BFUs. Thus it changes the order of last loop to handle this problem. A design methodology for conflict-free memory mapping scheme considering both arbitrary radix of in-place FFT and different number of parallel PEs is proposed in [8]. But this method requires $O(kb^2)$-bit look up tables for radix-b NTT with k PEs and needs a lot of buffers positioning at every stage for reordering. [13] propose the conflict-free addressing scheme for arbitrary radix FFT with multipath delay commutator (MDC)-based architecture but it consumes double memory storage. [10] applies extra registers to buffer and synchronize the data points fetched at different time slot, which leads to pipeline stalls and degrades the hardware utilization rate. In general, to our best knowledge, an efficient CFMMS considering both arbitrary radix in-place NTT and multiple number of BFUs is still in demand. In this paper, the primary contributions are summarized as belows:

1) We offer the general loop structure of arbitrary-radix in-place NTT. The loop unroll method is adopted to obtain the parallel NTT algorithm. Two point-fetching modes called as inter-group prioritization (IEP) and intra-group prioritization (IAP) are proposed according to data dependency.
2) We revise the linear transformation technique to build CFMMS for IEP-mode access. As for the IAP-mode, we transform the conflict-free memory mapping problem into conflict graph and develop a novel heuristic to explore the design space of CFMMS, which generates more efficient access schemes.
3) To verify the proposed access schemes, we present fully pipelined NTT/INTT hardware cores with different parallel degrees for Dilithium, whose area-time performance largely outperforms the advanced works. We also evaluate the hardware cost of CFMMS with different vector length N, radix R and parallel degree d under two point-fetching modes, respectively. The proposed IAP-mode CFMMS consumes smaller LUT overhead than the IEP-mode.

2 BACKGROUND

2.1 NTT Algorithm and Memory Conflict

In lattice-based PQC schemes, NTT over the ring is described as $A_i = \sum_{j=0}^{N-1} a_j \phi_{2N}^{j} \omega_N^{ij} \bmod q \quad i = 0, 1, ..., N-1$. NTT with in-place property enables data points to be stored into the location where they are read from. Thus the required memory footprint is equal to N-word vector length and concurrent read-write operation can be performed in a single clock cycle if we use the dual-port block RAM on FPGA. The classic DIT in-place NTT receives the vector data in

DAC '22, July 10–14, 2022, San Francisco, CA, USA
© 2022 Association for Computing Machinery.
ACM ISBN 978-1-4503-9142-9/22/07…$15.00
https://doi.org/10.1145/3489517.3530656

Authorized licensed use limited to: National Chung Hsing Univ.. Downloaded on February 23,2026 at 10:45:23 UTC from IEEE Xplore. Restrictions apply.

bit-reversed order and produces the result in natural order. But the bit-reversed cost can be avoided by rearranging the loop structure and adjusting the twiddle factors as illustrated in algorithm 1.

Algorithm 1: Radix-2 DIT NTT Algorithm

Input: $\boldsymbol{a}$, q, ϕ_{2N}.Here $\boldsymbol{a}$ denotes a vector of length N, q is modulus and ϕ_{2N} is $2N$-th primitive roots of unity.
Output: $\boldsymbol{A} = NTT(\boldsymbol{a})$

```
c ← 1
for p = log_2 N − 1 to 0 do
    J ← 2^p
    ω_m ← φ_{2N}^{N/(2J)}
    for k = 0 to N/(2J) − 1 do
        ω ← ω_m^{reverse(c)}
        c ← c + 1
        for j = 0 to J − 1 do
            s ← a_{2kJ+j}
            t ← ω · a_{2kJ+j+J}
            A_{2kJ+j} ← (s + t) mod q
            A_{2kJ+j+J} ← (s − t) mod q
return A
```

In the interleaved memory system, if radix-R in-place NTT contains d BFUs, $d \times R$ independent banks can be used to match the bandwidth of parallel access. The multi-bank based NTT hardware architecture is depicted in Figure 1, where memory mapper and interconnection modules guarantee conflict-free parallel access. Figure 1 also depicts the occurrence of memory conflict when applying memory mapping scheme of [6] to 8-point radix-2 in-place NTT with two BFUs. Obviously, the logic addresses {0,4,1,5} at stage 2 group 0 are mapped on to the bank indexes {0,1,1,2}, which results in memory conflict in bank 1. The similar situation also happens in bank 3 at stage 2 group 1.

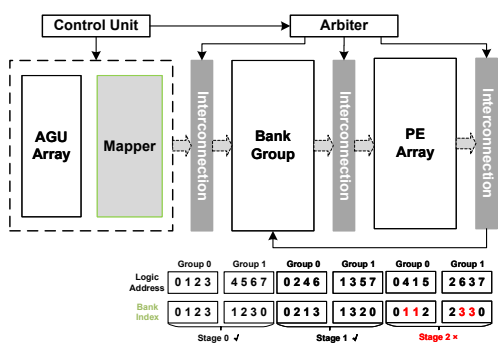

Figure 1: Multi-bank NTT architecture and memory conflict.

2.2 Strided Access Scheme

The access scheme is regarded as a function mapping n-bit logic address a into m-bit bank index BI and t-bit row address RA ($t = n - m$). The most simple access scheme is the low order interleaving as described in Figure 2 (a), where BI and RA are obtained by directly extracting the field from logic address. This scheme is suitable for linear access but performs poorly in stride or other types of access pattern. [2] proposes the row rotation (skewed) scheme to support the constant stride access as Figure 2 (b) shows. This scheme suffers from delay inherent in the adder and depending on the vector length. But it cannot backup the multi-stride access such as rows, columns and diagonals in the matrix access. The linear transformation technique (LTT) is introduced in [5]. Figure 2 (c) puts forward an example of mapping process for 5-bit logic address. This scheme is realized with bit-wise XOR and shift operation and alleviates the delay of adder. The binary transformation matrices R and T are used to obtain BI and RA, respectively.

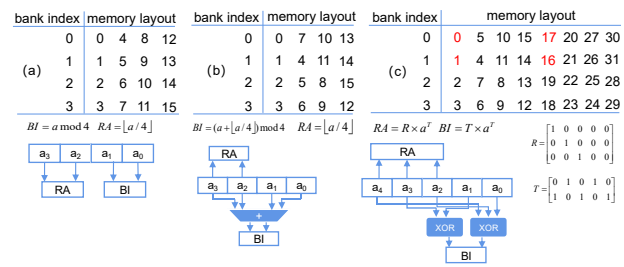

Figure 2: Memory layout in three types of access scheme.

3 GENERAL PARALLEL IN-PLACE NTT

The dataflow of 8-point radix-2 in-place NTT is depicted in Figure 3. We use the symbol $D^{0/1}_{p,k,j}$ to denote the data point in stage p group k and round j so that the data points denoted with the same color belong to the same group. As illustrated in algorithm 1, the number of iteration in middle loop k and innermost loop j both depend on the outer loop p but they are irrelevant with each other themselves. Thus, two types of point-fetching mode can be obtained by swapping the execution sequence of loop k and j.

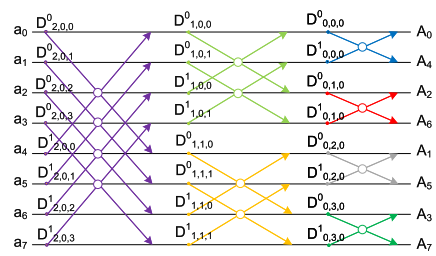

Figure 3: The dataflow of 8-point radix-2 DIT_{nr} NTT.

In-place NTT with abitrary radix. We focus on the loop structure of in-place NTT with arbitrary radix to analyze the generation principle of address. Inspired by the divide-and-conquer strategy to decimate DFT in time domain, we can derive the Cooley Tukey DIT NTT algorithm for arbitrary radix R in the following. First, we divide the NTT formula into R coresidual groups according to the time domain index j as:

$$A_i = \sum_{j=0}^{N/R-1} a_{Rj}\phi_{2N}^{Rj}\omega_N^{i\cdot(Rj)} + \sum_{j=0}^{N/R-1} a_{Rj+1}\phi_{2N}^{Rj+1}\omega_N^{i\cdot(Rj+1)} + \ldots + \sum_{j=0}^{N/R-1} a_{Rj+R-1}\phi_{2N}^{Rj+R-1}\omega_N^{i\cdot(Rj+R-1)} \bmod q \quad i = 0, 1, \ldots, N-1 \tag{1}$$

Then, by applying the elimination property $\omega_{dN}^{dk} = \omega_N^k$ and $\phi_{2dN}^{dk} = \phi_{2N}^k$, equation 1 can be transformed as:

$$A_i = \sum_{j=0}^{N/R-1} a_{Rj}\phi_{2N/R}^{j}\omega_{N/R}^{ij} + \omega_N^i \cdot \phi_{2N}^1 \cdot \sum_{j=0}^{N/R-1} a_{Rj+1}\phi_{2N/R}^{j}\omega_{N/R}^{ij} + \ldots + \omega_N^{(R-1)i} \cdot \phi_{2N}^{R-1} \cdot \sum_{j=0}^{N/R-1} a_{Rj+R-1}\phi_{2N/R}^{j}\omega_{N/R}^{ij} \bmod q \quad i = 0, 1, \ldots, N/R-1 \tag{2}$$

Authorized licensed use limited to: National Chung Hsing Univ.. Downloaded on February 23,2026 at 10:45:23 UTC from IEEE Xplore. Restrictions apply.

By using the periodicity property of twiddle factors $\omega_N^{k+N} = \omega_N^k$, we can obtain the R N/R-point NTT as belows:

$$A_{i+\frac{mN}{R}} = \sum_{k=0}^{R-1} \omega_N^{ki} \phi_{2N}^k \omega_R^{km} \cdot \sum_{j=0}^{N/R-1} a_{Rj+k} \phi_{2N/R}^j \omega_{N/R}^{ij} \bmod q \tag{3}$$
$$i = 0, 1, ..., N/R-1 \quad m = 0, 1, ..., R-1$$

Following this recursive method ahead until N/R R-point NTT, the DIT NTT algorithm with arbitrary radix can be obtained.

IAP point-fetching mode. The IAP-mode indicates that the data points within the same group k are fetched before the points of group $k+1$ in the parallel butterfly operations. Taking Figure 3 as an example, the IAP point-fetching mode at stage 1 is expressed in the following order: $\{D^0_{1,0,0}, D^1_{1,0,0}, D^0_{1,0,1}, D^1_{1,0,1}\} \rightarrow \{D^0_{1,1,0}, D^1_{1,1,0}, D^0_{1,1,1}, D^1_{1,1,1}\}$. The index k with red color stands for the group. Note that the number of iteration in middle loop k and innermost loop j depends on the first loop p. Thus the relative size between stride J and parallel degree d will influence the number of iteration if we apply the loop unroll technique in the in-place NTT. Eventually, the loop unroll under IAP mode is classified into two cases: (1) $J < d$, the parallel access will contain several groups and include the iterative range of inner loop j. (2) $J \geq d$, each group will cover several parallel accesses. The detailed parallel rdix-R in-place NTT under IAP mode is shown in algorithm 2, where the specific BF operation is left out while the generation of address is kept.

Algorithm 2: Parallel Radix-R NTT under IAP mode

```
Input: a, N, R. Here a denotes a vector of length N. d is the parallel degree. R denotes
       the radix of NTT.
Output: A = IAP_NTT(a)
for p = log_R N − 1 to 0 do
    J ← R^p;
    if J < d then
        for k = 0 to N/(Rd) − 1 do
            for i = 0 to d/J − 1 do
                for j = 0 to J − 1 do
                    t ← kRd + iRJ + j
                    [A_0, A_1, ..., A_{R−1}] = BF([a_t, a_{t+J}, ..., a_{t+(R−1)J}])
    else
        for k = 0 to N/(RJ) − 1 do
            for i = 0 to J/d − 1 do
                for j = 0 to d − 1 do
                    t ← kRJ + id + j
                    [A_0, A_1, ..., A_{R−1}] = BF([a_t, a_{t+J}, ..., a_{t+(R−1)J}])
return A
```

IEP point-fetching mode. The IEP-mode refers that the data points of group $k+1$ is fetched prior to the points of group k, which is exactly opposite to the IAP mode. Specifically, the order of reading point is defined as: $\{D^0_{1,0,0}, D^1_{1,0,0}, D^0_{1,1,0}, D^1_{1,1,0}\} \rightarrow \{D^0_{1,0,1}, D^1_{1,0,1}, D^0_{1,1,1}, D^1_{1,1,1}\}$. The index j with red color belongs to the same round. Obviously, this can be obtained by scheduling the loop j in advance of the loop k. In analogy with IAP mode, the loop unroll is also classified into two cases: (1) $J \geq N/(Rd)$, the innermost loop k will contain d rounds of butterfly operation. (2) $J < N/(Rd)$, each round will cover d groups of butterfly operation. As a result, the detailed parallel radix-R in-place NTT under IEP mode is presented in algorithm 3, which retains the method of address generation.

Algorithm 3: Parallel Radix-R NTT under IEP mode

```
Input: a, N, R. Here a denotes a vector of length N. d is the parallel degree. R denotes
       the radix of NTT.
Output: A = IEP_NTT(a)
for p = log_R N − 1 to 0 do
    J ← R^p;
    if J > N/Rd then
        for j = 0 to J/d − 1 do
            for k = 0 to N/(RJ) − 1 do
                for i = 0 to d − 1 do
                    t ← kRd + jd + i
                    [A_0, A_1, ..., A_{R−1}] = BF([a_t, a_{t+J}, ..., a_{t+(R−1)J}])
    else
        for j = 0 to J − 1 do
            for k = 0 to N/(RdJ) − 1 do
                for i = 0 to d − 1 do
                    t ← kRJd + iRJ + j
                    [A_0, A_1, ..., A_{R−1}] = BF([a_t, a_{t+J}, ..., a_{t+(R−1)J}])
return A
```

4 CONFLICT-FREE MEMORY MAPPING

4.1 CFMMS for Parallel NTT under IEP Mode

Problem formulation. We make two assumptions that vector length $N = 2^n$ and number of bank $M = 2^m$. As shown in section 2.2, the binary transformation matrix R and $T_{N,M}$ are used to obtain the row address and bank index, respectively. The matrix R usually consists of $t \times t$-size identity matrix with ones filled in the main diagonal so that the row address is obtained by extracting the $(n-t)$ most significant bits of address a. It is suggested in [4] that the matrix $T_{N,M}$ is full rank to ensure the unique storage location for every logic address. Thus, $T_{N,M}$ is usually composed of ones in the main diagonal as well. But this condition still cannot lead to the conflict-free access in NTT as Figure 2 (c) shows.

Analysis and solution. The reason for the occurrence of such a memory conflict is twofold: (1) The last stage in NTT will contain both stride-1 and stride-$N/2$ access while the other stages only contain one stride-2^p access. (2) If the total number of stage n meets that $n \bmod m \neq 0$, then there will be an incomplete diagonal left after filling the consecutive $m \times m$-size main diagonal with ones. To handle this problem, additional ones should be added into matrix $T_{N,M}$ if $n \bmod m \neq 0$. [11] presents an solution but it gives no explanation. We point out that this scheme can be obtained by filling the matrix $T_{N,M}$ with diagonal ones starting from the right lower corner. If the left space of matrix for diagonal is not available, then a partial diagonal is placed and the rest partial diagonal wraps back to the right. If it will reach the top row of matrix, then the wrapping will proceed in the next column. As a result, $(n + m - gcd(q, n \bmod m))$ ones will be added in transformation matrix $T_{2^n,2^m}$, where $gcd(\cdot)$ denotes the greatest common denominator. Then the entire access scheme is formalized as belows:

$$\begin{aligned} RA &= R \times a^{\mathrm{T}} = a_{n-1}a_{n-2}...a_{n-m} \\ BI_i &= T_{N,M} \times a^{\mathrm{T}} = \bigoplus_{k=0}^{h_{n,m}(i)} a_{(k \cdot m+i) \bmod n}, \quad i = 0, 1, ..., m-1 \\ h_{n,m}(i) &= \lceil (n + m - gcd(m, n \bmod m) - i - 1)/m \rceil \end{aligned} \tag{4}$$

Authorized licensed use limited to: National Chung Hsing Univ.. Downloaded on February 23,2026 at 10:45:23 UTC from IEEE Xplore. Restrictions apply.

Taking 32-point radix-2 in-place NTT with two BFUs as an example, the binary transformation matrix for bank index becomes $T_{32,4}$ and the detailed mapping process is shown in Figure 4 (I). In this way, the logic addresses {0,16,1,17} at the first round of the last stage are mapped onto four different bank indexes {0,1,3,2}, respectively. The remaining point-fetching process also avoids the memory conflict completely.

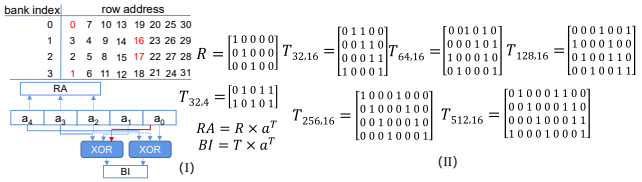

Figure 4: Memory layout and transformation matrix.

Hardware model. In the practical implementation, the number of memory bank M is a design time parameter while vector length N is varying with the application. Figure 4 (II) depicts the variation of transformation matrix $T_{N,M}$ with N when M is fixed to 16. The structure of off-diagonal has a periodic behavior and the period is m. The concrete hardware architecture of address mapper for varying vector length is shown in Figure 5, which consists of bit-wise XORs, ANDs and left rotation unit. The address a is divided

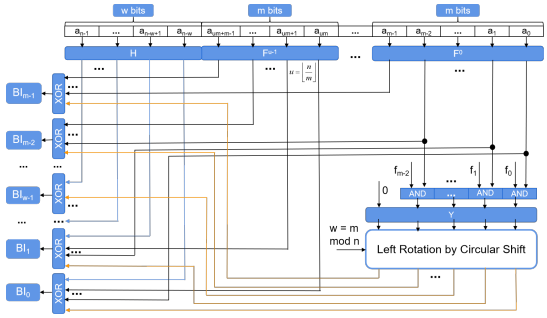

Figure 5: Hardware design of configurable address mapper.

into several m-bit fields starting from the least significant bit , i.e., $F^k = (a_{km+m-1}, a_{km+m-2}, ..., a_{km+1}, a_{km})^{\mathrm{T}}$, where $k = 0, 1, ..., u$ and $u = \lceil F/m \rceil$. Then the remaining w most significant bits are extracted as a bit vector H, i.e., $H = (a_{n-1}, a_{n-2}, ..., a_{n-w})^{\mathrm{T}}$. This vector only participates in the least w XOR operations to obtain BI_0 to BI_{w-1}, which are denoted with blue lines. The structure of off-diagonals for different vector length indicates the rotation of the least $m - gcd(m, w)$ significant bits. Thus, the m-bit vector $Y = (0, ..., 0, a_{m-gcd(m,w)-1}, ..., a_1, a_0)^{\mathrm{T}}$ is obtained by assigning $gcd(m, w)$ zeroes in $(f_0, f_1, ..., f_{m-2})$ for truncation based on ANDs. Then m-bit rotation vector $R = (r_{m-1}, ..., r_1, r_0)^{\mathrm{T}}$ is obtained by performing w-bit left circular shift of vector Y. The rotation vector R will take part in the whole bit-wise XOR operations, which are denoted with yellow lines. To conclude, the bit-wise XOR operations between vector H,a and R are formalized as belows. Obviously, each individual XOR gate will only take at most $u + 2$ fan-ins.

$$BI_i = \begin{cases} r_i \oplus (\bigoplus_{k=0}^{u-1} a_{km+i}) & i \geq w \\ h_i \oplus r_i \oplus (\bigoplus_{k=0}^{u-1} a_{km+i}) & i < w \end{cases} \tag{5}$$

4.2 CFMMS for Parallel NTT under IAP Mode

Problem formulation. Recalling that the IAP mode will contain concurrent stride-1 and stride-N/R access at every stage. As Figure 2 (c) depicts, the logic addresses {0,4,1,5} are mapped onto the bank indexes {0,1,0,1} so that two concurrent accesses should be performed on each bank, which results in the so-called memory conflict. To tackle this problem, we try to transform the conflict-free mapping problem into conflict graph. We regard each logic address as vertex and bridge the concurrently accessed addresses with edges. Then we map the given number of banks onto each vertex in the way that the connected vertexes are assigned with different bank indexes. Figure 6 (I) presents two examples of conflict graph for 8-point radix-2 in-place NTT, where the memory banks are denoted with different colors. However, the searching complexity of this conflict graph is proportional to the vector length N and parallel degree d. This makes the exploration of valid solution really difficult especially for the larger vector length.

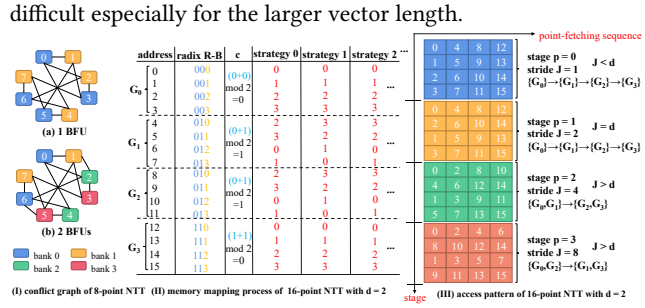

Figure 6: Conflict graph and CFMMS of 16-point radix-2 NTT.

Analysis and solution. To reduce the searching complexity, we aim to leverage the toggling principle of the address-bit stream of in-place NTT. We develop new heuristic to figure out valid solutions based on the following observations. (1) **Coresidual check.** Taking algorithm 1 as an example, we make a closer look at the form of two addresses at each round, i.e., $2kJ+j$ and $2kJ+j+J$. It is observed that two addresses are both function of indexes k, j, J, while the range of index j, k is from 0 to $J-1(=2^p-1)$ and 0 to $2^{n-p-1}-1$, respectively. Thus, the address can be expressed as a bit vector, i.e., $a = (a_{n-1}, a_{n-2}, ..., a_{p+1}, a_p, a_{p-1}, ..., a_0)^{\mathrm{T}}$. The difference between two addresses is embodied by address bit a_p, which is fixed to 0 for the smaller address and 1 for the bigger one. Furthermore, the position of address bit a_p is varying with the stage of NTT. Then the coresidual check is obtained by performing modular accumulation on the bit vector: $c = (a_{n-1}+a_{n-2}+...+a_{p+1}+a_p+a_{p-1}+...+a_0) \bmod 2$. The value of c is essentially influenced by a_p so that two addresses will always be different in terms of c. There will be a similar principle in the higher radix R in-place NTT and a_p ranges from 0 to $R-1$ at that time, which corresponds to R different logic addresses in each round of NTT. (2) **Performing permutation.** We take the parallel access into consideration so that there will be $B = R \times d$ banks to match the bandwidth. We will show how the conflict-free memory mapping is achieved by valid permutations based on the coresidual check. Taking 16-point radix-2 NTT with two BFUs ($d = 2$) as an example, the detailed mapping process is shown in Figure 6 (II). The 16 data points are allocated into 4 groups by cyclic partition. If concurrently accessed data points are all just from a single group, four different banks can be easily assigned to each point in arbitrary sequence without any conflict. But the data points will be fetched from different groups when stride J is greater than the parallel degree d as Figure 6 (III) shows. Thus, the four banks should be arranged in an appropriate permutation to avoid the mapping collision between different groups. In fact, R types of permutation

Authorized licensed use limited to: National Chung Hsing Univ.. Downloaded on February 23,2026 at 10:45:23 UTC from IEEE Xplore. Restrictions apply.

are enough to meet this requirement since the data points will be fetched from at most R different groups in the radix-R in-place NTT under the IAP mode. Another important observation is that

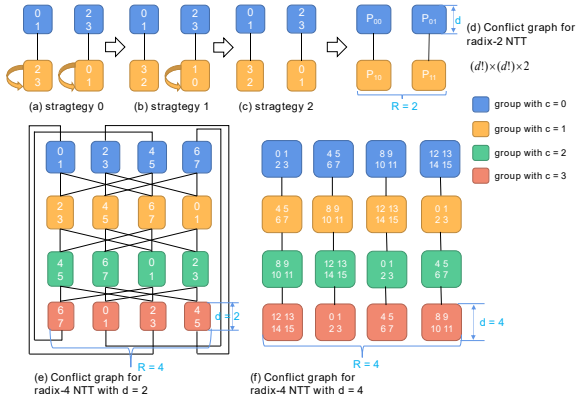

Figure 7: Conflict graph based on the access pattern of groups.

the value of c in these related R groups will be exactly different from each other. Hence, the selection of permutation types for each group can be made according to c. The logic address is divided into two parts, i.e., a_H and a_L, so that the least $\log_r(R \times d)$ significant bits a_L are transformed in radix-B. The remaining most significant bits a_H are expressed in radix-r ($r^k = R$). The value of c is obtained by the modular accumulation of the bit vector in a_H at this time. The fourth to sixth columns present three different valid strategies for two types of permutation. (3) **Developing the conflict graph.** We transform the conflict-free memory mapping problem between data points into the one between groups based on item (2) and (3). In other words, the research on the access pattern for N data points is transformed into the one for R types of permutation, which largely reduces the searching complexity. Then we will show why these listed strategies are valid and how to devise an appropriate strategy. Figure 7 presents the conflict graph based on the access pattern of groups, which describes the conflict between different groups in the parallel access of NTT. As mentioned in item (2), conflict is likely to occur when data points from several different groups are accessed simultaneously and the value of c will be different among these related groups. Therefore, groups can be further classified into R types according to c. For strategy 0 depicted in Figure 7 (a), two types of permutation are selected so that four different memory banks are allocated into the connected blue group and yellow group without conflict. Rearranging the position of bank indexes within each colored groups will not influence the conflict-free situation. Thus, strategy 1 and 2 can be easily derived by reordering the bank indexes within yellow groups as shown in Figure 7 (b) and (c). Figure 7 (e) presents the general form of conflict graph for radix-2 NTT, which is composed of 2×2 vertex array and each vertex will contain d different bank indexes. As a result, the search space for valid permutation only depends on the radix R and parallel degree d, which is irrelevant with N. By derivation, there will be at least $2 \times d!^2$ valid solutions for the radix-2 in-place NTT. Figure 7 (e) and (f) present the conflict graph for radix-4 NTT with $d = 2$ (8 BFUs) and $d = 4$ (16 BFUs), respectively. The connection in Figure 7 (e) is more complicated than Figure 7 (f) since radix-4 NTT with $d = 2$ will cover two different groups in stage 1 and four different groups in the remaining stages. While the radix-4 NTT with $d = 4$ always cover four different groups. One can derive the conflict graph in terms of other radices and parallel degrees based this method.

Proposed heuristic and hardware model. We formalize the above illustration into a heuristic to devise the efficient CFMMS as algorithm 4 shows. The general hardware model for the scheme

Algorithm 4: CFMMS for radix-r^k IAP-mode NTT.

Input: a, N, r^k, d. Here r is a prime and $r^k = R$ denotes the radix of NTT.
Output: BI, RA.
Step 1. Let B denote the number of banks, where B meets that $B = R \times d$ and $N \mid B$; The digit width T of a expressed in radix r is $T = log_r N$; The digit width of B expressed in radix r is $M = log_r B = log_r(R \times d)$; Then a can be expressed in radix r as:
$a = [a_{T-1}, a_{T-2}, ..., a_1, a_0]_r = [a_{T-1}, ..., a_M, a_{M-1}, ..., a_1, a_0]_r$;
Step 2. Expressing $a_L = [a_{M-1}, ..., a_1, a_0]_r$ in radix B as $[b]_B$;
Expressing $a_H = [a_{T-1}, ..., a_M]_r$ in radix R as $[b_{m-1}, b_{m-2}, ..., b_0]_R$;
Performing coresidual check: $c = (b_{m-1} + b_{m-2} + ... + b_0) \bmod R$;
Step 3. a_L is used to obtain the permutations. The valid R types of permutation for each group, i.e., $PM_0, PM_1, ..., PM_{R-1}$, are determined according to conflict graph;
switch (c) :
$0 : BI = PM_0(a_L)$
$1 : BI = PM_1(a_L)$
...
$R-1 : BI = PM_{R-1}(a_L)$
end switch
Step 4. The row address is calculated as: $RA = [a_{T-1}, ..., a_M]_r$;
return BI, RA.

is presented in Figure 8 (a), where modular accumulation unit is used to perform coresidual check and PM unit is composed of digit level gates to realize the valid R types of permutation. Figure 8

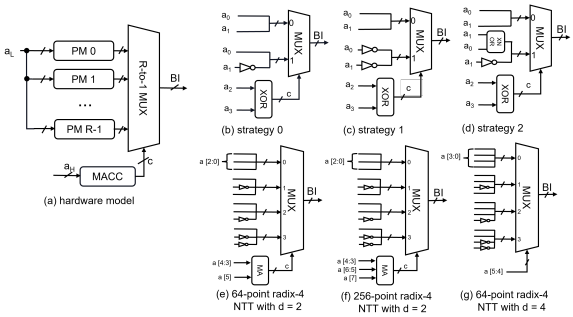

Figure 8: Hardware model of CFMMS under IAP mode.

(b) to (d) depict the hardware implementation of strategy 0 to 2, which implies that strategy 0 is the most lightweight implementation among these strategies. Figure 8 (e) to (g) present the hardware architecture for different N and d. It is observed that increasing the vector length will only influence the length of modular accumulation chain while the parallel degree only has an effect on the width of input to PM unit. Thus, this architecture can easily support the configuration for different N and d.

4.3 Case Study of NTT/INTT for Dilithium

Dilithium is one of the third-round PQC signature schemes based on Module-LWE problem. In NIST Level 3 parameter setting, 5 NTT and 6 INTT operations for 256-degree polynomial are required to perform the matrix-vector multiplication, which dominates the main computation overhead of the entire algorithm. As case study to verify the above access schemes, we design the multi-bank NTT/INTT hardware kernel with different parallel degrees to strike sound balance between area and latency.

Design Rationale. Several optimized techniques similar to [15] [14] are applied to the hardware architecture, including low-complexity NTT/INTT, bit-reversed-free loop structure, reusing

Authorized licensed use limited to: National Chung Hsing Univ.. Downloaded on February 23,2026 at 10:45:23 UTC from IEEE Xplore. Restrictions apply.

the twiddle factors and unified butterfly units. Besides, we offer the improved Barrett reduction method to implement modular multiplier as illustrated in algorithm 5. Based on the property of modulus $q = 2^{23} - 2^{13} + 1$, we reduce the number of 23-bit multiplication from 3 to 1 by adopting hardware-friendly shift and addition operations.

Algorithm 5: Improved Barrett reduction for Dilithium.

Input: $q = 2^{23} - 2^{13} + 1, z = x \cdot y, k, \mu$. Here z is the 46-bit product. Quotient $k = ((z >> 22) \cdot \mu) >> 24 = z[45:23] + z[45:33] + z[45:44] + z[45]$.
24-bit $\mu = \lceil \frac{2^{46}}{q} \rceil = 2^{23} + 2^{13} + 2 + 1$.
Output: $t = z \bmod q$

```
a = z[33:23] + z[43:33] + z[45:44] + z[45]
b = a + (z[45:34] << 11)
c = z[23] + z[33] + z[44] + z[45]
t = (z[23:0] − k · q) mod q = z[23:0] − (c << 23) + (b << 13) − a
if t > q then t = t − q
return t
```

5 IMPLEMENTATION AND COMPARISON

Table 1: NTT comparison with related works.

Works	Cycle	Freq.(MHz)	#BFs	LUT/FF/DSP/BRAM	Time(μs)
[16](A)	1170	216	2	2044 / - / 16 / 6	5.4
[9](U)	512	637	2	2236 / 2532 / 48 / 3.5	0.8(NTT)
[9](U)	512	637	2	2547 / 3889 / 84 / 3.5	0.8(INTT)
[7](A)	533/536	142	2	524 / 759 / 17 / 1	3.7
TW(U)	1308	447	1	506 / 485 / 2 / 1.5	2.9
	536	420	2	1276 / 945 / 4 / 2.5	1.27
	270	408	4	2922 / 1805 / 8 / 5	0.66
TW(A)	1308	211	1	493 / 485 / 2 / 1.5	6.2
	536	204	2	1256 / 945 / 4 / 2.5	2.6
	270	202	4	2877 / 1800 / 8 / 5	1.3

Comparison of NTT hardware core. We use Verilog HDL to implement these designs on both Artix-7 (xc7z020clg484-3) and Virtex UltraScale+ (xcvu7p-flvb2104-2-i) FPGA for fair comparison. The resource usage and highest frequency are obtained from Vivado 2020.2 as table 1 shows. [9] presents the separated NTT and INTT designs with 2 BFs, which achieves the highest frequency at the price of vast resource consumption. [16] consumes both 1.6× more resources and 2.0× larger latency than our 2-BF design. [7] utilizes 4.25× more DSPs to compensate 2.4× less LUTs compared with our 2-BF design. It also consumes 1.5× latency than ours. The proposed access schemes make NTT/INTT kernels fully pipelined, which achieves nearly optimal scalability and area-time performance.

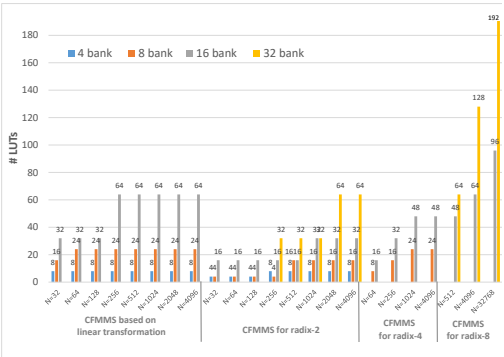

Figure 9: The LUT overhead of CFMMS.

Figure 9 depicts the LUT overhead for CFMMS with different N and d. The proposed IAP-mode CFMMS consumes smaller LUT overhead than that of IEP mode under the same N and d. For example, when setting $N = 1024$ and $\#bank = 16$, the LUT consumption of LTT is 64 while the radix-2 and radix-4 CFMMS consumes 32 LUTs and 48 LUTs, respectively. The reason for this difference is that LTT requires multiple XOR chains to implement IEP-mode CFMMS while the proposed MUX-based CFMMS only consumes one XOR chain to perform the coresidual check. This difference is enlarged when increasing N and d.

6 CONCLUSION

We aim to solve memory collision issue for generalized two point-fetching modes. Besides revising the LTT, we firstly propose the conflict graph based heuristic to explore more efficient access schemes. We also make case study of NTT/INTT designs for Dilithium, which achieves much higher performance compared to state-of-the-arts. We hope that the graph-based memory allocation methodology also facilitates other schemes for complex stride access.

ACKNOWLEDGMENTS

We thank the editors and reviewers for their thoughtful comments. This work is supported in part by the National Key RD Program of China (Grant No. 2018YFB2202101), and in part by the National Natural Science Foundation of China (Grant No. 62104129, Grant No. 61834002), and in part by the Beijing Superstring Academy of Memory Technology, and in part by the Jiangsu Province Science Foundation for Youths (BK20170205) and in part by the provincial Key Research and Development Programs (BE2021032, BE2020016).

REFERENCES

[1] Banerjee, U., Ukyab, T. S., and Chandrakasan, A. P. Sapphire: A configurable crypto-processor for post-quantum lattice-based protocols. *IACR Trans. Cryptogr. Hardw. Embed. Syst. 2019*, 4 (2019), 17–61.
[2] Budnik, P., and Kuck, D. J. The organization and use of parallel memories. *IEEE Trans. Computers 20*, 12 (1971), 1566–1569.
[3] Feng, X., and Li, S. Accelerating an FHE integer multiplier using negative wrapped convolution and ping-pong FFT. *IEEE Trans. Circuits Syst. II Express Briefs 66-II*, 1 (2019), 121–125.
[4] III, D. T. H. Block, multistride vector, and FFT accesses in parallel memory systems. *IEEE Trans. Parallel Distributed Syst. 2*, 1 (1991), 43–51.
[5] III, D. T. H. Increased memory performance during vector accesses through the use of linear address transformations. *IEEE Trans. Computers 41*, 2 (1992).
[6] Johnson, L. Conflict free memory addressing for dedicated fft hardware. *IEEE Transactions on Circuits and Systems II: Analog and Digital Signal Processing* (1992).
[7] Land, G., Sasdrich, P., and Güneysu, T. A hard crystal - implementing dilithium on reconfigurable hardware. In *CARDIS 2021* (2021), vol. 13173 of *Lecture Notes in Computer Science*, Springer, pp. 210–230.
[8] Reisis, D. I., and Vlassopoulos, N. Conflict-free parallel memory accessing techniques for FFT architectures. *IEEE Trans. Circuits Syst. I Regul. Pap.* (2008).
[9] Ricci, S. Implementing crystals-dilithium signature scheme on fpgas. In *ARES 2021, Vienna, Austria, August 17-20* (2021), ACM.
[10] Roy, S. S., and Vercauteren, F. Compact ring-lwe cryptoprocessor. In *Cryptographic Hardware and Embedded Systems - CHES* (2014).
[11] Takala, J., Jarvinen, T., and Sorokin, H. Conflict-free parallel memory access scheme for fft processors. In *ISCAS* (2003).
[12] Wang, W., Huang, X., Emmart, N., and Weems, C. C. VLSI design of a large-number multiplier for fully homomorphic encryption. *IEEE Trans. Very Large Scale Integr. Syst. 22*, 9 (2014), 1879–1887.
[13] Xia, K., Wu, B., Xiong, T., and Ye, T. A memory-based FFT processor design with generalized efficient conflict-free address schemes. *IEEE Trans. Very Large Scale Integr. Syst. 25*, 6 (2017), 1919–1929.
[14] Xing, Y., and Li, S. A compact hardware implementation of cca-secure key exchange mechanism CRYSTALS-KYBER on FPGA. *IACR Trans. Cryptogr. Hardw. Embed. Syst. 2021*, 2 (2021), 328–356.
[15] Zhang, N., Yang, B., Chen, C., Yin, S., Wei, S., and Liu, L. Highly efficient architecture of newhope-nist on FPGA using low-complexity NTT/INTT. *IACR Trans. Cryptogr. Hardw. Embed. Syst. 2020*, 2 (2020), 49–72.
[16] Zhou, Z. A software/hardware co-design of crystals-dilithium signature scheme. *ACM Trans. Reconfigurable Technol. Syst. 14*, 2 (2021), 11:1–11:21.

Authorized licensed use limited to: National Chung Hsing Univ.. Downloaded on February 23,2026 at 10:45:23 UTC from IEEE Xplore. Restrictions apply.